

Simulation Engine for Computational Vector Calculus, Kinematic Tracing, and Error-Bounded Differentiation

Koen Hicswa

Independent Research

May 2026

Abstract

This paper presents the Vector Field Simulation with Physics Integration framework, an interactive visual and computational engine designed for 2D vector field analysis. The software evaluates path independence, field conservativeness, and localized vorticity. Visualizations are expanded upon by introducing a component-wise multi-solver kinematic particle tracking engine, real-time discrete line integration along freehand or parameterized paths, and a asymptotic error profiling of the threshold constraints of the Central Difference Method.

Keywords: Vector Calculus, Numerical Integration, Runge-Kutta, Central Differences, Computational Physics.

1 Introduction

The visualization of vector-valued functions is a massive component of multi-variable calculus. But mapping complex non-uniform force fields by hand is tedious and impractical for analysis.

To address this challenge, this simulation framework was developed for rapid visualization and real-time numerical analysis. This framework contributes to student learning by lowering the barrier to deep analysis without requiring massive mathematical engines or expensive licenses.

2 System Architecture and Implementation

The engine uses a modular architecture that separates parsing, numerical processing, and graphic rendering.

2.1 Software Stack

The development framework leverages three core libraries:

- **Dear ImGui:** Handles user input, and displays outputs to the user.
- **ExprTk:** A mathematical string parsing and evaluation engine that compiles user-defined component strings into bytecode at runtime.
- **SFML:** Manages the main render window and drawing calls.

2.2 Coordinate Transformation Mapping

To translate user interactions on the SFML render window back to mathematical space, the program converts pixel coordinates $(x_{\text{screen}}, y_{\text{screen}})$ on a viewport $W \times H$ into a coordinate space defined by the values G_x and G_y .

$$x_{\text{math}} = \left(\frac{x_{\text{screen}}}{W} \right) \cdot G_x - \frac{G_x}{2} \quad (1)$$

$$y_{\text{math}} = - \left(\frac{y_{\text{screen}} - H}{H} \right) \cdot G_y - \frac{G_y}{2} = \left(1 - \frac{y_{\text{screen}}}{H} \right) \cdot G_y - \frac{G_y}{2} \quad (2)$$

2.3 Execution Pipeline

The user must input $P(x, y)$ and $Q(x, y)$. The process follows:

1. **Validation Phase:** User strings are evaluated via the bytecode compiler. If structural or syntax validation fails, an alert popup notifies the user.
2. **Discretization Phase:** Upon validation, parameters are sampled across uniform coordinate intervals bounded by $[-G_x/2, G_x/2]$ and $[-G_y/2, G_y/2]$ to build a collection of **Vector2D** objects tracking localized positions, components, magnitude, and direction angles.
3. **Analytical Phase:** The verification module runs, solving local partial derivatives across the space to evaluate field conservativeness and curl vectors.

3 Mathematical Foundations & Core Upgrades

A field is formalized as a mapping over a subset of two-dimensional Euclidean space:

$$\vec{F}(x, y) = \langle P(x, y), Q(x, y) \rangle \quad (3)$$

For practical purposes, rendered vector orientations are normalized via a scaling and clamping function. The rendering magnitude L is calculated using a scaling scalar $\sigma = 12.0$ and clamped within a pixel range of $[10.0, 30.0]$:

$$L = \max \left(10.0, \min \left(30.0, \sigma \sqrt{P^2 + Q^2} \right) \right) \quad (4)$$

The screen rendering direction vector $\vec{d}_{\text{screen}}$ accounts for the inverted y -axis of screen space relative to standard Cartesian coordinate using the calculated angle $\theta = \text{atan2}(Q, P)$:

$$\vec{d}_{\text{screen}} = \langle \cos(\theta), -\sin(\theta) \rangle \quad (5)$$

Field vector colors are determined based on a range of magnitudes $\|\vec{F}\|$:

$$\text{Color}(\|\vec{F}\|) = \begin{cases} \text{Cyan} & \text{if } \|\vec{F}\| < 0.5 \\ \text{Green} & \text{if } 0.5 \leq \|\vec{F}\| < 1.5 \\ \text{Yellow} & \text{if } 1.5 \leq \|\vec{F}\| < 3.0 \\ \text{Red} & \text{if } \|\vec{F}\| \geq 3.0 \end{cases} \quad (6)$$

3.1 Vorticity and Programmatic Curl

The curl of a two-dimensional vector field evaluates the net rotational density around a localized perpendicular axis:

$$\text{curl}(\vec{F}) = \frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \quad (7)$$

In standard 3D space, this scalar represents the $\hat{k}$ -component of a 3D curl vector. However, for formatting purposes (`CurlX`, `CurlY`, `CurlZ`), the engine packages this scalar value into the first index slot of a three-dimensional tuple:

$$\vec{\omega}_{\text{engine}} = \left\langle \left(\frac{\partial Q}{\partial x} - \frac{\partial P}{\partial y} \right), 0.0, 0.0 \right\rangle \quad (8)$$

If $|\omega_x| \leq \epsilon_{\text{threshold}}$ (where $\epsilon_{\text{threshold}} = 0.005$) across all sampled coordinate vectors within the grid domain, the engine identifies the field as conservative, guaranteeing the existence of a scalar potential field ϕ such that $\nabla\phi = \vec{F}$. Under these conditions, the work performed along any closed contour C satisfies:

$$\oint_C \vec{F} \cdot d\vec{r} = 0 \quad (9)$$

3.2 Asymptotic Derivative Error Profiling

To evaluate partial derivatives, the framework implements a central difference algorithm using an adjustable step size h . The engine isolates coordinate paths to evaluate the shifted components:

$$\frac{\partial Q}{\partial x} \approx \frac{Q(x+h, y) - Q(x-h, y)}{2h} \quad (10)$$

$$\frac{\partial P}{\partial y} \approx \frac{P(x, y+h) - P(x, y-h)}{2h} \quad (11)$$

The total computational error $E(h)$ is modeled as a combination of two numerical constraints:

1. **Truncation Error** (ϵ_{trunc}): Derived directly via the Taylor series expansions of a multi-variable function about point y :

$$P(x, y+h) = P(x, y) + h \frac{\partial P}{\partial y} + \frac{h^2}{2} \frac{\partial^2 P}{\partial y^2} + \frac{h^3}{6} \frac{\partial^3 P}{\partial y^3} + \mathcal{O}(h^4) \quad (12)$$

$$P(x, y-h) = P(x, y) - h \frac{\partial P}{\partial y} + \frac{h^2}{2} \frac{\partial^2 P}{\partial y^2} - \frac{h^3}{6} \frac{\partial^3 P}{\partial y^3} + \mathcal{O}(h^4) \quad (13)$$

Subtracting equation (13) from (12) eliminates the even-order derivative terms:

$$P(x, y+h) - P(x, y-h) = 2h \frac{\partial P}{\partial y} + \frac{h^3}{3} \frac{\partial^3 P}{\partial y^3} + \mathcal{O}(h^5) \quad (14)$$

Dividing by $2h$ isolates the derivative term, demonstrating that the truncation error scales with $\mathcal{O}(h^2)$:

$$\frac{P(x, y+h) - P(x, y-h)}{2h} = \frac{\partial P}{\partial y} + \frac{h^2}{6} \frac{\partial^3 P}{\partial y^3} + \mathcal{O}(h^4) \implies \epsilon_{\text{trunc}} = \frac{h^2}{6} \left| \frac{\partial^3 P}{\partial y^3} \right| \quad (15)$$

2. **Round-off Error** (ϵ_{round}): Double-precision floating-point arithmetic imposes a precision limit ($\epsilon_{\text{mach}} \approx 2.2 \times 10^{-16}$). Subtracting near-identical values results in a loss of significance scaling with $\mathcal{O}(\frac{\epsilon_{\text{mach}}}{h})$:

$$\epsilon_{\text{round}} \approx \frac{\epsilon_{\text{mach}} \cdot |P(x, y)|}{h} \quad (16)$$

The complete numerical error profile is defined as:

$$E(h) = C_1 h^2 + C_2 \frac{\epsilon_{\text{mach}}}{h} \quad (17)$$

Differentiating $E(h)$ with respect to h and setting the result to zero minimizes error:

$$\frac{dE}{dh} = 2C_1 h - C_2 \frac{\epsilon_{\text{mach}}}{h^2} = 0 \implies h_{\text{opt}} = \sqrt[3]{\frac{C_2 \cdot \epsilon_{\text{mach}}}{2C_1}} \approx 10^{-5} \quad (18)$$

The software includes an slider that allows users to observe different values of $\vec{\omega}$ if h is configured above 10^{-3} (truncation dominant) or below 10^{-12} (round-off dominant).

3.3 Component-Wise Kinematic Particle Solvers

To analyze vector spaces dynamically, a tracer module tracks state variables by treating the field as an implicit velocity map where $\frac{d\vec{r}}{dt} = \vec{F}(\vec{r}(t))$.

1. **Euler's Method (First-Order)**: Maps immediate forward approximations:

$$x_{n+1} = x_n + P(x_n, y_n) \Delta t \quad (19)$$

$$y_{n+1} = y_n + Q(x_n, y_n) \Delta t \quad (20)$$

2. **Midpoint Method (Second-Order Runge-Kutta / RK2)**: Evaluates a half-step projection state to eliminate first-order drift constraints:

$$x_{\text{mid}} = x_n + P(x_n, y_n) \frac{\Delta t}{2}, \quad y_{\text{mid}} = y_n + Q(x_n, y_n) \frac{\Delta t}{2} \quad (21)$$

$$x_{n+1} = x_n + P(x_{\text{mid}}, y_{\text{mid}}) \Delta t \quad (22)$$

$$y_{n+1} = y_n + Q(x_{\text{mid}}, y_{\text{mid}}) \Delta t \quad (23)$$

3. **Classical Runge-Kutta (RK4 - Fourth-Order)**: Establishes stable orbital trajectories by combining four distinct weighted vector updates:

$$\begin{aligned} k_{1x} &= P(x_n, y_n), & k_{1y} &= Q(x_n, y_n) \\ k_{2x} &= P\left(x_n + k_{1x} \frac{\Delta t}{2}, y_n + k_{1y} \frac{\Delta t}{2}\right), & k_{2y} &= Q\left(x_n + k_{1x} \frac{\Delta t}{2}, y_n + k_{1y} \frac{\Delta t}{2}\right) \\ k_{3x} &= P\left(x_n + k_{2x} \frac{\Delta t}{2}, y_n + k_{2y} \frac{\Delta t}{2}\right), & k_{3y} &= Q\left(x_n + k_{2x} \frac{\Delta t}{2}, y_n + k_{2y} \frac{\Delta t}{2}\right) \\ k_{4x} &= P(x_n + k_{3x} \Delta t, y_n + k_{3y} \Delta t), & k_{4y} &= Q(x_n + k_{3x} \Delta t, y_n + k_{3y} \Delta t) \end{aligned} \quad (24)$$

The structural components are updated using a weighted average:

$$x_{n+1} = x_n + \frac{\Delta t}{6} (k_{1x} + 2k_{2x} + 2k_{3x} + k_{4x}) \quad (25)$$

$$y_{n+1} = y_n + \frac{\Delta t}{6} (k_{1y} + 2k_{2y} + 2k_{3y} + k_{4y}) \quad (26)$$

Tracer trails are handled via a container limited to size S_{trail} . Alpha fading channels are calculated dynamically across history indexes i :

$$\alpha(i) = \left\lfloor 255.0 \cdot \left(1.0 - \frac{i}{S_{\text{trail}}}\right) \right\rfloor \quad (27)$$

3.4 Upgrade C: Real-Time Discrete Line Integration Loops

To evaluate path independence, users can choose between dynamic parametric string generation $(x(t), y(t))$ across domain boundaries $[t_{\text{start}}, t_{\text{end}}]$ with configured intervals Δt_{param} or freehand path plotting on the grid canvas.

The integration module computes total work done by gathering sequential vertex tuples $C = \langle \vec{r}_0, \vec{r}_1, \dots, \vec{r}_N \rangle$. It tracks explicit directional displacement variables ($\Delta x_i = x_{i+1} - x_i$, $\Delta y_i = y_{i+1} - y_i$) and maps them to a Midpoint Riemann Sum:

$$\int_C \vec{F} \cdot d\vec{r} \approx \sum_{i=0}^{N-1} \vec{F} \left(\frac{\vec{r}_i + \vec{r}_{i+1}}{2} \right) \cdot \Delta \vec{r}_i \quad (28)$$

This expression is expanded inside the computational loops:

$$W = \sum_{i=0}^{N-1} \left[P \left(\frac{x_i + x_{i+1}}{2}, \frac{y_i + y_{i+1}}{2} \right) \Delta x_i + Q \left(\frac{x_i + x_{i+1}}{2}, \frac{y_i + y_{i+1}}{2} \right) \Delta y_i \right] \quad (29)$$

4 Empirical Testing Suite & Results

The engine was validated across three baseline test fields. Table 1 tracks the computational properties observed during application runtime.

Table 1: Empirical Test Field Configurations and Observed System Behaviors

Field Components	Topology Profile	Curl Vector ($\vec{\omega}_{\text{engine}}$)	Line Integral (W)
$P = \sin(y), Q = \sin(x)$	Confinement Cells	$\langle \cos(x) - \cos(y), 0.0, 0.0 \rangle$	Non-Zero
$P = x, Q = -y$	Hyperbolic Slopes	$\langle 0.0, 0.0, 0.0 \rangle$	0.00000
$P = \cos(x + y), Q = \sin(x - y)$	Wave Patterns	$\langle -\sin(x - y) + \sin(x + y), 0.0, 0.0 \rangle$	Non-Zero

When testing the conservative properties of Field 2 ($P = x, Q = -y$) along user-defined closed paths, the computed line integral returned a value of 0.00000 (within a tolerance boundary matching ϵ_{mach}), confirming path independence. Testing Field 1 over closed paths intersecting cell boundaries yielded values away from zero, capturing the macroscopic behavior of non-zero curl.

5 Conclusion

The expanded Simulation Engine provides a workspace for computational vector calculus research. By adding kinematic solvers, line integration loops, and step-size optimization controls, the framework shifts from a visualization package into an educational tool for validating computational physics.

References

- [1] A. Partow, “The C++ Mathematical Expression Toolkit Library (ExprTk),” *Partow Technologies*, 2020.
- [2] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997.
- [3] E. Hairer, S. P. Nørsett, and G. Wanner, *Solving Ordinary Differential Equations I: Nonstiff Problems*, Springer, 1993.
- [4] R. L. Burden and J. D. Faires, *Numerical Analysis*, Cengage Learning, 2011.